

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia (E)				
TÍTULO DE LA PRÁCTICA:	<i>Transform Quest - Juego de Puzzles con Transformaciones Geométricas 2D</i>				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	
FECHA DE PRESENTACIÓN		HORA DE PRESENTACIÓN			
INTEGRANTE (s): - Nina Suyo Diego Claudio				NOTA:	
DOCENTE(s): <i>Mg. Diego Alonso Iquira Becerra</i>					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Link del Github: https://github.com/DiegoNiS/CGVCM_Lab02-TransformQuest I. OBJETIVOS - Conocer, comprender y crear aplicaciones multimedia que utilicen transformaciones geométricas 2D. - Implementar las transformaciones afines elementales traslación, rotación y escalado en un videojuego 2D desarrollado en Unity 6. - Aplicar las transformaciones geométricas de forma interactiva respondiendo a las acciones del jugador. II. MARCO TEÓRICO Transformaciones Geométricas Afines Las transformaciones geométricas afines son aquellas que conservan la colinealidad de puntos y la razón de distancias. Se expresan matricialmente como: $P' = f(P) = AP + W$ Donde A es una matriz 2×2 (parte lineal) y W es el vector de traslación. Las transformaciones afines elementales son la traslación, la rotación y el escalado.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Traslación

La traslación desplaza un punto $P = (x, y)$ sumando un vector $w = (a, b)$:

$$x' = x + a$$

$$y' = y + b$$

En Unity, la traslación se aplica modificando la propiedad `transform.position` o usando `Rigidbody2D.MovePosition()`. En este proyecto se usa para mover el personaje y el fantasma enemigo.

Rotación

La rotación gira un punto un ángulo θ alrededor del origen:

$$x' = x \cdot \cos(\theta) - y \cdot \sin(\theta)$$

$$y' = x \cdot \sin(\theta) + y \cdot \cos(\theta)$$

En Unity se aplica mediante `transform.Rotate()`. En este proyecto se usa para animar las pociones (rotación continua) y la palanca (rotación al activarse).

Escalado

El escalado modifica las dimensiones de un objeto según factores p (eje X) y q (eje Y):

$$x' = p \cdot x$$

$$y' = q \cdot y$$

En Unity se aplica mediante `transform.localScale`. En este proyecto se usa para encoger y agrandar al personaje al recoger las pociones.

III. DESARROLLO DE LA APLICACIÓN

3.1 Descripción General

Se desarrolló 'Transform Quest', un juego de puzzles 2D con vista top-down en Unity 6. El jugador controla un personaje en una mazmorra y debe resolver una secuencia de puzzles usando transformaciones geométricas para llegar al cofre del tesoro antes de que se acabe el tiempo (50 segundos).

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3



3.2 Mecánica del Juego

El juego sigue esta secuencia de puzzles:

- El personaje empieza en tamaño normal en la esquina superior izquierda.
- Debe recoger la Poción Verde para encogerse a la mitad de su tamaño (Escalado) y poder pasar por el pasillo estrecho.
- Debe evitar al Fantasma que patrulla la zona central (Traslación).
- Debe recoger la Poción Azul para regresar a su tamaño normal (Escalado inverso).
- Debe activar la Palanca con ESPACIO para abrir la Puerta (Rotación).
- Finalmente llega al Cofre del tesoro para ganar.

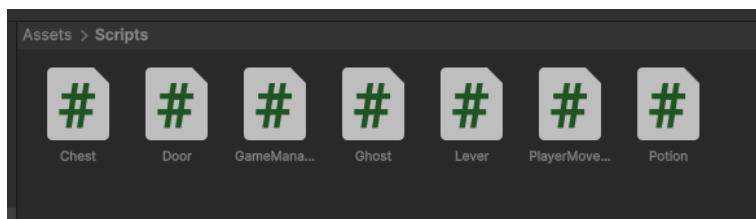
Si el tiempo se agota o el fantasma atrapa al jugador, el juego termina. Se puede reiniciar con la tecla R y el mejor tiempo se guarda entre partidas.

3.3 Estructura del Proyecto

- Scripts/GameManager.cs — Cronómetro, condiciones de victoria/derrota y mejor tiempo
- Scripts/PlayerMovement.cs — Movimiento del personaje, volteo de sprite y estado (normal/pequeño)
- Scripts/Potion.cs — Rotación continua y efecto de escalado al ser recogida
- Scripts/Lever.cs — Detección de proximidad y rotación al activarse

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

- Scripts/Door.cs — Cambio de sprite y desactivación de colisión al abrirse
- Scripts/Ghost.cs — Traslación automática entre dos puntos con cambio de dirección
- Scripts/Chest.cs — Detección de llegada del jugador y activación de victoria



3.4 Explicación del Código

Transformación 1: Escalado — Potion.cs y PlayerMovement.cs

Las pociones rotan continuamente aplicando una rotación sobre el eje Z en cada frame:

```
CSharp
void Update()
{
    // ROTACIÓN continua de la poción
    transform.Rotate(0f, 0f, rotationSpeed * Time.deltaTime);
}
```

Al ser recogida, la poción aplica escalado al jugador. La poción verde lo encoge a la mitad:

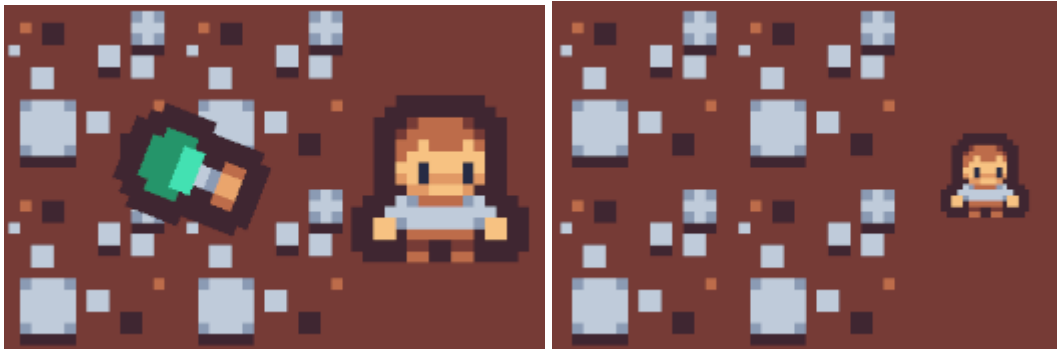
```
CSharp
public void Shrink()
{
    isSmall = true;
    // ESCALADO: reduce el tamaño a 0.5 en X e Y
    transform.localScale = new Vector3(0.5f, 0.5f, 1f);
}
```

La poción azul lo devuelve a su tamaño original:

```
CSharp
public void Grow()
{
    isSmall = false;
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```
// ESCALADO inverso: restaura tamaño normal
transform.localScale = new Vector3(1f, 1f, 1f);
}
```



Transformación 2: Rotación — Lever.cs

La palanca detecta cuando el jugador está cerca y al presionar ESPACIO (solo en tamaño normal) inicia una rotación animada de 90 grados:

CSharp

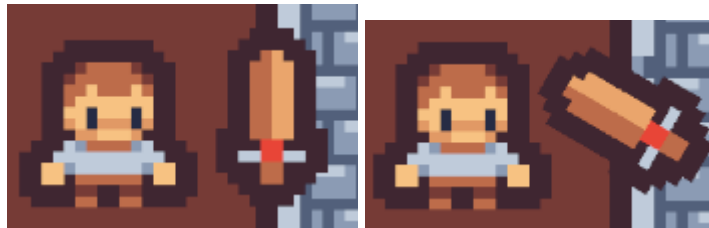
```
if (isRotating)
{
    float step = rotationSpeed * Time.deltaTime;
    float remaining = rotationAmount - currentRotation;
    if (step >= remaining)
    {
        transform.Rotate(0f, 0f, remaining);
        isRotating = false;
        door.Open(); // Abre la puerta al terminar
    }
}
else
{
    // ROTACIÓN progresiva frame a frame
    transform.Rotate(0f, 0f, step);
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

        currentRotation += step;
    }
}

```



Transformación 3: Traslación — Ghost.cs y PlayerMovement.cs

El fantasma se traslada automáticamente entre dos puntos configurables, cambiando de dirección al llegar a cada extremo:

```

CSharp
void Update()
{
    float moveDir = movingRight ? 1f : -1f;
    // TRASLACIÓN: desplazamiento horizontal continuo
    transform.Translate(Vector2.right * moveDir * speed * Time.deltaTime);

    if (transform.position.x >= endX) movingRight = false;
    else if (transform.position.x <= startX) movingRight = true;
}

```

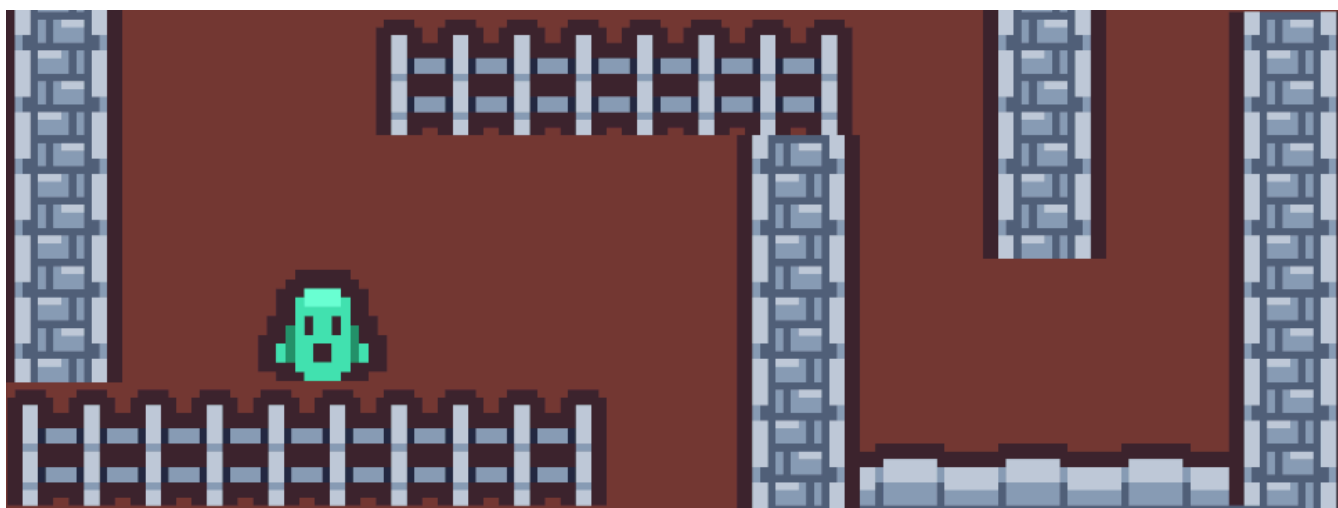
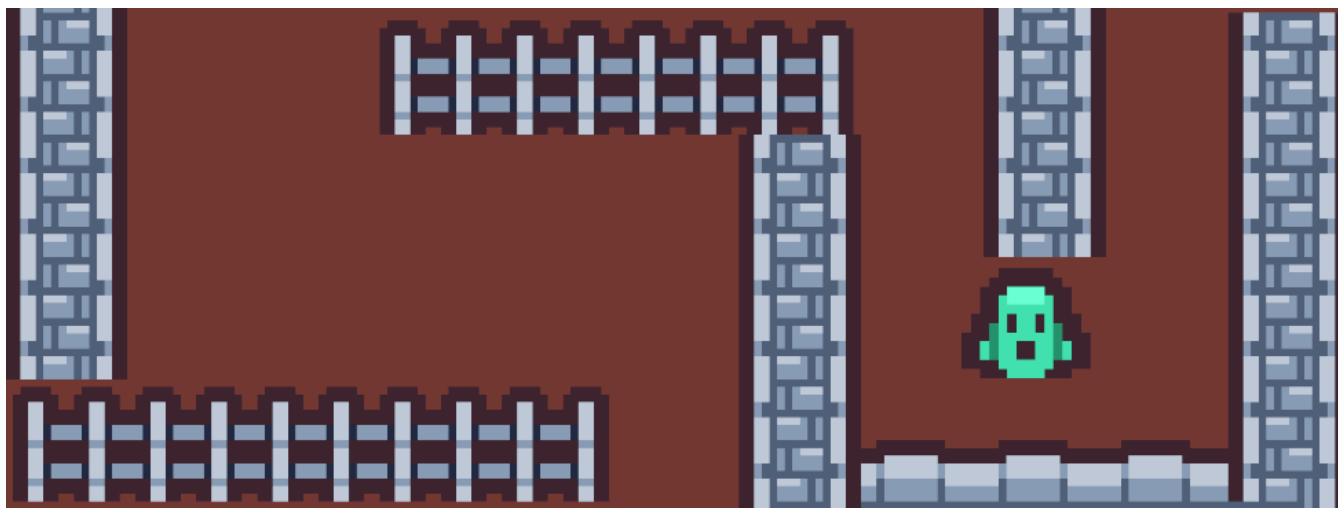
El personaje también usa traslación para su movimiento, controlado por el jugador:

```

CSharp
void FixedUpdate()
{
    float speed = isSmall ? smallSpeed : normalSpeed;
    // TRASLACIÓN: mueve al jugador según input
    rb.MovePosition(rb.position + movement * speed * Time.fixedDeltaTime);
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

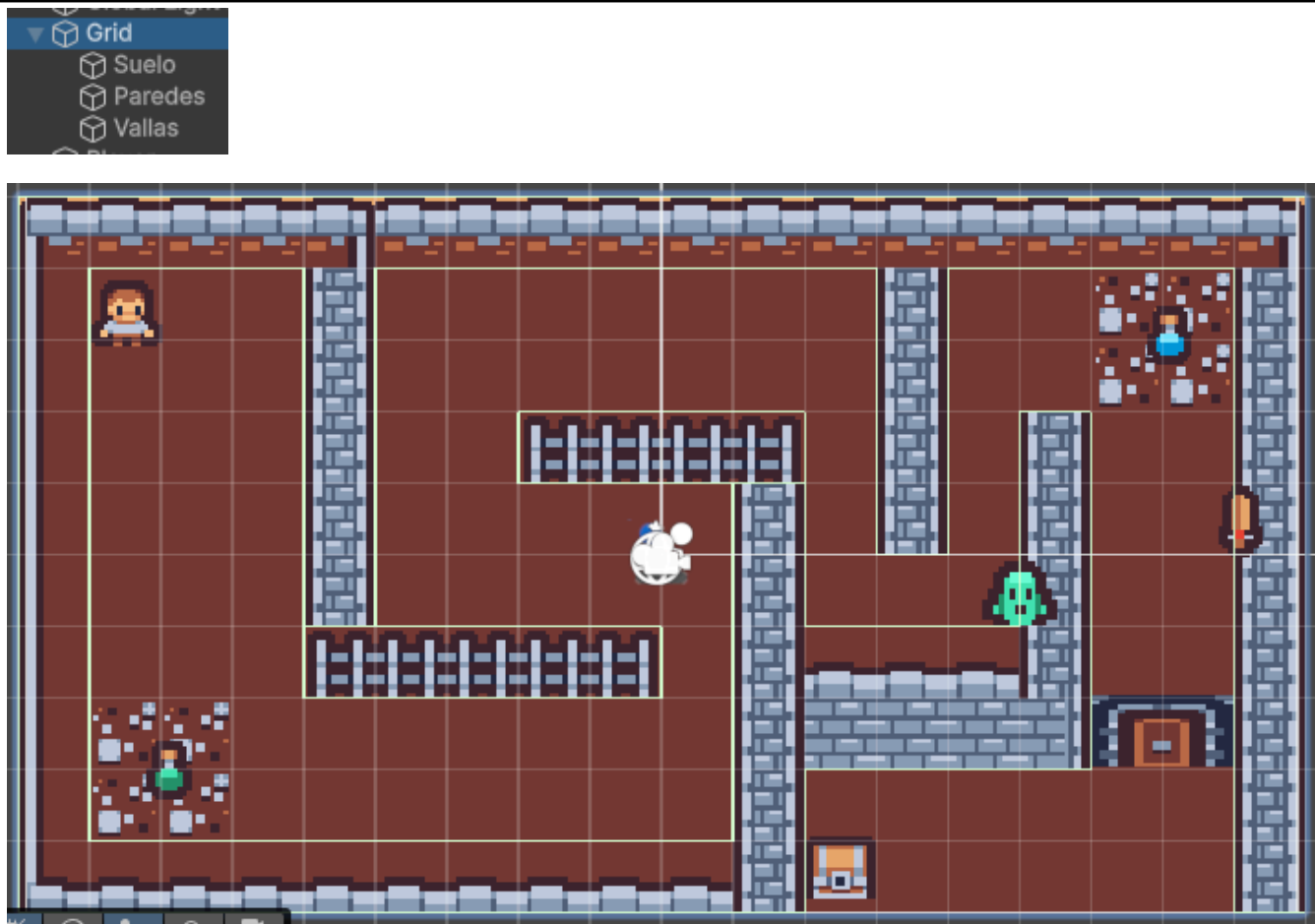


IV. ESCENARIO Y TILEMAP

El escenario fue construido usando el sistema de Tilemap de Unity con el asset gratuito Tiny Dungeon de Kenney.nl. Se crearon tres capas de Tilemap:

- Suelo: tiles de tierra marrón que cubren toda la habitación.
- Paredes: bloques de piedra gris que forman los muros exteriores e interiores dividiendo el espacio en zonas.
- Vallas: elementos decorativos de valla que complementan las divisiones internas.

Las paredes tienen TilemapCollider2D con CompositeCollider2D para generar colisiones eficientes. El Order in Layer se configuró para que los elementos se dibujen en el orden correcto: Suelo (0), Paredes/Vallas (1), Personaje y objetos (2).



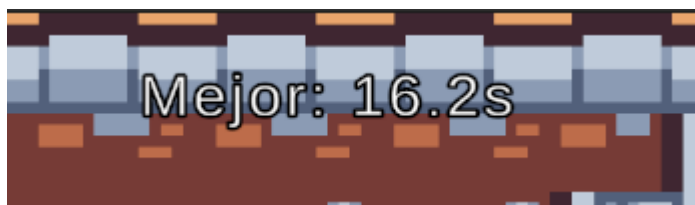
V. SISTEMA DE UI Y PUNTUACIÓN

La interfaz muestra:

- Cronómetro regresivo de 30 segundos — se vuelve rojo cuando quedan menos de 10 segundos.
- Mensaje de victoria o derrota al terminar el juego con el tiempo empleado.
- Mejor tiempo guardado entre partidas usando PlayerPrefs.
- Tecla R para reiniciar inmediatamente después de ganar o perder.

El mejor tiempo se guarda con `PlayerPrefs.SetFloat()` y persiste aunque se cierre el juego, mostrando siempre el récord personal del jugador.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

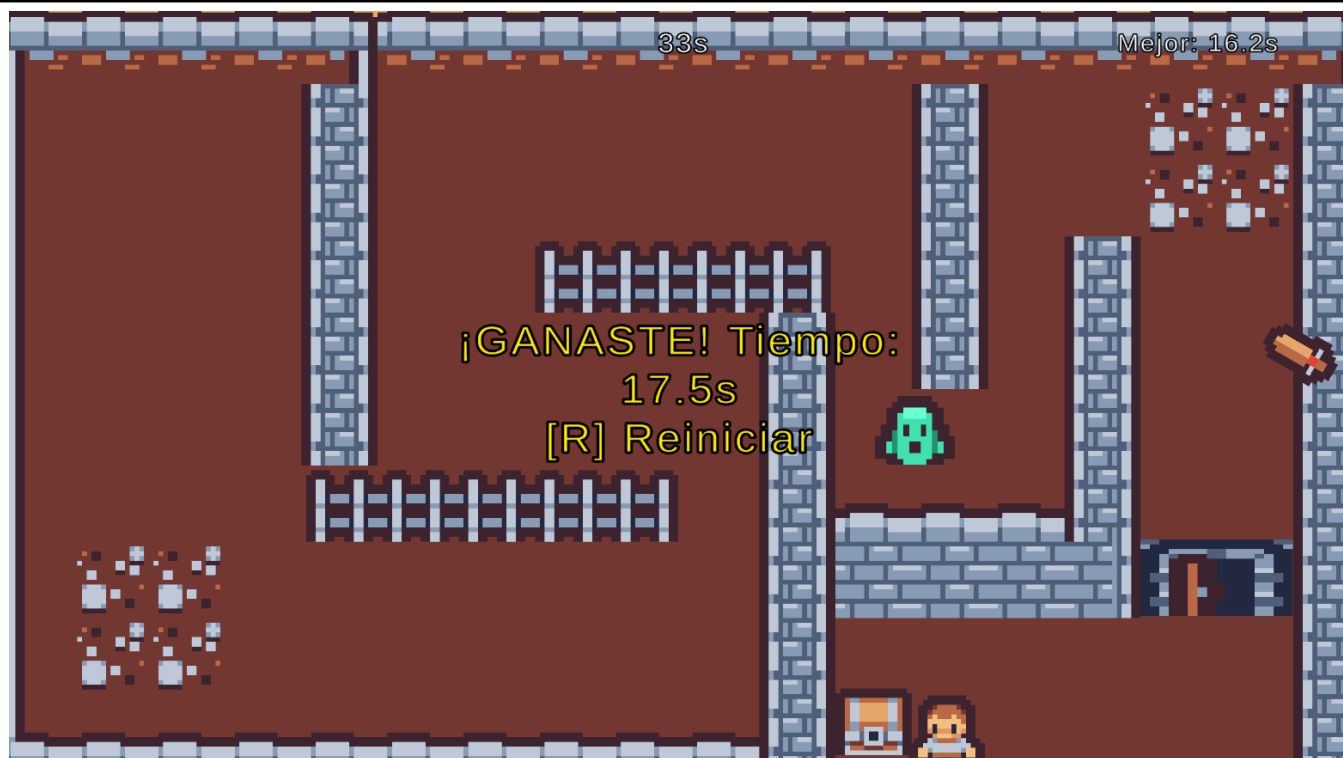


VI. RESULTADOS

Se logró desarrollar un juego de puzzles 2D completamente funcional que implementa las tres transformaciones geométricas del laboratorio:

- Traslación: movimiento del personaje y patrullaje del fantasma enemigo.
- Rotación: animación continua de las pociones y rotación de la palanca al activarse.
- Escalado: el personaje se encoge al recoger la poción verde y recupera su tamaño con la poción azul.
- Sistema de puzzles con secuencia lógica que obliga al jugador a aplicar las transformaciones en orden.
- Cronómetro de 30 segundos con condición de derrota y mejor tiempo guardado.
- Fantasma enemigo que elimina al jugador al contacto.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10



II. SOLUCIÓN DEL CUESTIONARIO

– No hay cuestionario –

III. CONCLUSIONES

El desarrollo de este laboratorio permitió comprender de forma práctica cómo las transformaciones geométricas afines se aplican en el desarrollo de videojuegos. Unity expone estas transformaciones a través de `transform.position` (traslación), `transform.Rotate()` (rotación) y `transform.localScale` (escalado), que internamente aplican las matrices afines estudiadas en el marco teórico.

La implementación del sistema de puzzles demostró que las transformaciones geométricas no solo tienen utilidad gráfica sino también mecánica: el escalado del personaje cambia su hitbox y velocidad, la rotación de la palanca activa eventos del juego, y la traslación del fantasma genera una amenaza dinámica que el jugador debe esquivar.

El uso del sistema de Tilemap de Unity con el pack Tiny Dungeon de Kenney.nl permitió construir un escenario de mazmorra coherente y visualmente atractivo sin invertir tiempo en la creación de arte propio.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*. Stylus Publishing, LLC.
- [2] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019). *Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction*.
- [3] Unity Technologies. (2024). *Unity 6 Documentation — Transform*.
<https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Transform.html>
- [4] Unity Technologies. (2024). *Unity 6 Documentation — Tilemap*.
<https://docs.unity3d.com/6000.0/Documentation/Manual/class-Tilemap.html>
- [5] Kenney.nl. (2024). *Tiny Dungeon Asset Pack*. <https://kenney.nl/assets/tiny-dungeon>